Internet Applications

Introduction to TypeScript

Suhel Hammoud

What Will We Learn?

- What TypeScript adds to JavaScript.
- How to give variables and functions types.
- How to describe users and assignments.
- How to find simple mistakes before running code.

Start with JavaScript You Know

```
let studentName = "Maya";
let score = 80;
let submitted = false;
```

- `"Maya"` is text: a **string**.
- `80` is a **number**.
- `false` is a **boolean**: true or false.

TypeScript uses these same JavaScript values.

A Small JavaScript Surprise

What will this print?

```
function addTen(score) {
  return score + 10;
}

console.log(addTen("80"));
```

It prints `8010`, not `90`.

The argument is text, so `+` joins the values as text.

TypeScript Can Catch This Mistake

```ts
function addTen(score: number) {
  return score + 10;
}

console.log(addTen(80)); // 90
// addTen("80");        // Type error
```

- `score: number` says: this function expects a number.
- The checker reports a mistake if we pass a string.
- A **type error** means the code does not match the expected type.

What Is TypeScript?

- TypeScript is JavaScript with a system for checking types.
- JavaScript syntax such as `if`, loops, and functions still works.
- TypeScript files use the `.ts` extension.
- Editors can show type errors while we write code.
- Type information is removed before the code runs.

Set Up a Practice Folder

Use a new folder. Node.js and npm should already be installed.

```
mkdir typescript-practice
cd typescript-practice
npm init -y
npm pkg set type=module
npm install --save-dev typescript tsx @types/node
mkdir src
```

- `typescript` checks our code.
- `tsx` runs our TypeScript files.
- `@types/node` helps TypeScript understand Node.js features.

Copy This Starter Configuration

Create a file named `tsconfig.json` in the practice folder:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "types": ["node"]
  },
  "include": ["src/**/*.ts"]
}
```

Keep `strict: true` so the checker helps us catch more mistakes.

We will use these settings without studying each one today.

📘 Your First TypeScript File

Create `src/index.ts`:

```
let studentName: string = "Maya";

console.log(studentName);
```

Read the first line as:

“Create a variable called studentName that holds a string.”

Run It, Then Check It

Run these commands from the practice folder:

```
npx tsx src/index.ts
```

This **runs** the program and prints `Maya`.

```
npx tsc --noEmit
```

This **checks** the types without creating JavaScript files.

No error messages from the checker means the type check passed.

Three Types to Start With

```
let courseName: string = "Internet Applications";
let score: number = 80;
let submitted: boolean = false;
```

- `string`: text, usually written inside quotes.
- `number`: numbers such as `80` or `8.5`.
- `boolean`: either `true` or `false`.

The part after `:` is called a **type annotation**.

Changing a Value

```ts
let score: number = 80;

score = 90; // OK: another number
// score = "90"; // Type error: this is a string
```

- A variable's value can change while keeping the same type.
- `90` and `"90"` have different types.
- Uncomment the last line and run `npx tsc --noEmit`.
- Read the error, then comment the line again.

TypeScript Can Guess the Type

```
let score = 80;

score = 90; // OK
// score = "90"; // Still a type error
```

- TypeScript sees the first value and works out the type.
- This is called **type inference**.
- We do not need to write `: number` every time.

Try It: Three Variables

In `src/index.ts`, create:

1. `assignmentTitle`: a string.
2. `maxScore`: a number.
3. `completed`: a boolean.

Print all three with `console.log()`.

Check your code, then run it:

```
npx tsc --noEmit
npx tsx src/index.ts
```

🟦 Give a Function an Input Type

```
function greet(name: string) {
  return "Hello, " + name;
}

console.log(greet("Maya")); // Hello, Maya
// greet(42);              // Type error
```

- `name` is the function's input, also called a **parameter**.
- `name: string` means the input must be text.

Give a Function an Output Type

```
function doubleScore(score: number): number {
  return score * 2;
}

console.log(doubleScore(20)); // 40
```

- `score: number` describes the input.
- `): number` describes the value returned by the function.
- TypeScript can also infer the output type from `return`.

Try It: A Simple Function

Write a function named `assignmentMessage`.

- It accepts one input: `title`, a string.
- It returns a string: `"Assignment: "` followed by the title.

Start here:

```
function assignmentMessage(title: string): string {
  // Write your return statement here.
}
```

`assignmentMessage("Practice types")` should return:

```
Assignment: Practice types
```

Arrays: A List of One Type

```
const scores: number[] = [70, 80, 90];

scores.push(100); // OK
// scores.push("100"); // Type error
```

- `number[]` means “an array of numbers.”
- `string[]` means “an array of strings.”

Try creating a `string[]` containing three student names.

Objects: Group Related Values

```
const student = {
  id: 1,
  name: "Maya",
};

console.log(student.name); // Maya
```

- An object groups values into named **properties**.
- Here, `id` is a number and `name` is a string.
- TypeScript can infer the types of these properties too.

Choose from a Few Allowed Values

A user in our course can be a student or an instructor.

```
type UserRole = "student" | "instructor";

let role: UserRole = "student";
role = "instructor"; // OK
// role = "visitor"; // Type error
```

- `type` gives a type a name we can reuse.
- `|` means “or.” This is called a **union**.
- Only the two listed strings are allowed here.

An Interface Describes an Object

```
interface User {
  id: number;
  name: string;
  role: UserRole;
}
```

Read this as:

“A User must have an id, a name, and a role.”

An **interface** describes the properties an object should have.

Create a User

Use the `UserRole` and `User` definitions from the previous slides:

```
const user: User = {
  id: 1,
  name: "Maya",
  role: "student",
};

console.log(user.name); // Maya
```

- `: User` asks the checker to check this object's properties.
- Removing `name` or using `id: "1"` causes a type error.

Describe an Assignment

```ts
interface Assignment {
  id: number;
  title: string;
  completed: boolean;
}

const assignment: Assignment = {
  id: 10,
  title: "Practice TypeScript",
  completed: false,
};
```

The same idea works for other objects in our application.

Some Properties Can Be Optional

The `?` means a property may be left out:

```
interface Lesson {
  title: string;
  description?: string;
}

const lesson: Lesson = { title: "TypeScript" };
```

- `title` is required.
- `description` is optional.
- If `description` is absent, reading it gives `undefined`.

Check an Optional Value Before Using It

Using the `lesson` from the previous slide:

```
if (lesson.description !== undefined) {
  console.log(lesson.description.toUpperCase());
} else {
  console.log("No description yet");
}
```

- Inside the `if`, TypeScript knows the description is a string.
- Using a check to learn a more specific type is **narrowing**.
- Strict checking reminds us to handle missing values.

any Turns Off Helpful Checks

```ts
let value: any = "Maya";

value = 42; // Allowed
// value.toUpperCase(); // Checker allows it; running it fails
```

- `any` allows operations without checking that they make sense.
- A number does not have a `toUpperCase()` method.
- Prefer a specific type when you know what a value should be.

unknown Means We Need to Check First

```ts
function printName(value: unknown) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  }
}

printName("Maya"); // Prints MAYA
printName(42);     // Prints nothing
```

- `unknown` means we do not yet know the value's type.
- `typeof` checks the type while the program runs.
- The check lets us safely use string methods inside the `if`.

Types Do Not Check Every Rule

```ts
function showScore(score: number) {
  console.log(score);
}

showScore(-10); // This passes the type checker.
```

- `-10` is a number, so it matches the type.
- A rule such as “scores must be positive” needs a code check.
- Data arriving from a user or an API also needs runtime checks.
- Adding a TypeScript annotation does not validate incoming data.

Optional: Wrap Data in an API Result

An API often returns data inside another object.

```
interface AssignmentResult {
  data: Assignment;
}

const result: AssignmentResult = {
  data: assignment,
};

console.log(result.data.title);
```

Here, `data` holds the assignment we created earlier.

Optional: One Result Type for Different Data

A **generic** lets us reuse a type with different data types:

```
interface ApiResult<T> {
  data: T;
}
```

- `T` is a placeholder for a type we will choose later.
- `ApiResult<User>` means `data` holds a `User`.
- `ApiResult<Assignment>` means `data` holds an `Assignment`.

For now, this result only describes successful responses.

Optional: Try the Generic Result

Use the `user` and `assignment` objects from earlier:

```
const userResult: ApiResult<User> = {
  data: user,
};

const assignmentResult: ApiResult<Assignment> = {
  data: assignment,
};

console.log(userResult.data.name);
console.log(assignmentResult.data.title);
```

Try wrapping your assignment array in `ApiResult<Assignment[]>`.

References

Types (https://www.typescriptlang.org/docs/handbook/2/everyday-types.html)

Narrowing (https://www.typescriptlang.org/docs/handbook/2/narrowing.html)

Generics (https://www.typescriptlang.org/docs/handbook/2/generics.html)

tsx (https://tsx.hirok.io/typescript)